

Android + Firebase + ESP32 External LED Control System

Android and Kotlin with MVP | Firebase cloud communication | ESP32 with an external LED

Objective

Each group must develop a working prototype in which an authenticated user can control an external LED connected to an ESP32 using an Android application.

Expected Communication Flow

Android Application → Firebase → ESP32 → External LED

ESP32 → Firebase → Android Application

The ESP32 must send its actual LED status back to Firebase so that the status can be displayed in the Android application.

Hardware Requirements

- The same ESP32 board used in the previous activity
- One external LED
- One 220-ohm or 330-ohm resistor
- Breadboard
- Jumper wires
- USB data cable

Required System Functions

Android Application

1. Authenticate the user using Firebase Authentication.
2. Provide controls for turning the external LED ON and OFF.
3. Write the selected command to Firebase.
4. Display the command requested by the user.
5. Display the actual LED status reported by the ESP32.
6. Display whether the ESP32 is online, offline, or unavailable.
7. Display appropriate loading, success, and error messages.
8. Receive status changes from Firebase in real time.

ESP32

1. Connect to the configured Wi-Fi network.
2. Connect to Firebase using the approach introduced in the previous activity.
3. Retrieve the LED command from Firebase.
4. Turn the external LED on or off based on the received command.
5. Send the actual LED state back to Firebase.
6. Report a device status or last-update value.
7. Attempt to reconnect when the Wi-Fi connection is interrupted.

MVP Architecture Requirements

The Android application must follow the Model-View-Presenter pattern.

Component	Required Responsibility
View	Displays controls, LED status, connection status, and messages.
Presenter	Receives user actions and coordinates application operations.
Model or Repository	Handles Firebase Authentication and database operations.

The Activity or Fragment must not contain all Firebase and business logic directly.

Development Procedure

1. Identify the exact ESP32 board used by your group.
2. Select and document a safe output-capable GPIO pin.
3. Connect the external LED with a current-limiting resistor.
4. Test the LED using a basic blinking program.
5. Reconnect the ESP32 to Wi-Fi.
6. Configure the Firebase database structure.
7. Program the ESP32 to retrieve LED commands.
8. Program the ESP32 to report its actual LED status.
9. Implement the Android application using MVP.
10. Test the complete Android-Firebase-ESP32 communication flow.

Important: Do not proceed to Firebase troubleshooting until the external LED passes the local blinking test.

Required Test Scenarios

1. The Android application sends an ON command, and the LED turns on.
2. The Android application sends an OFF command, and the LED turns off.
3. The ESP32 reports its actual LED state to Firebase.
4. The Android application displays the status reported by the ESP32.
5. The ESP32 is disconnected from Wi-Fi.
6. The application properly handles an unavailable or offline ESP32.
7. The ESP32 reconnects and resumes communication.
8. An invalid or unavailable Firebase command does not cause the ESP32 program to crash.

Demonstration Requirements

- Log in to the Android application.
- Turn the external LED on and off.
- Show the corresponding Firebase values.
- Show the actual device status in the application.
- Demonstrate what happens when Wi-Fi is disconnected.
- Explain the complete data flow.
- Identify the View, Presenter, and Model components.
- Explain the GPIO pin and circuit used.

Any group member may be asked to explain or demonstrate any part of the system.

Important Reminder

This is a group deliverable, but every member must understand how the Android application, Firebase, ESP32, and external LED work together. ***Proper documentation of individual contributions is required.***